

Introducción

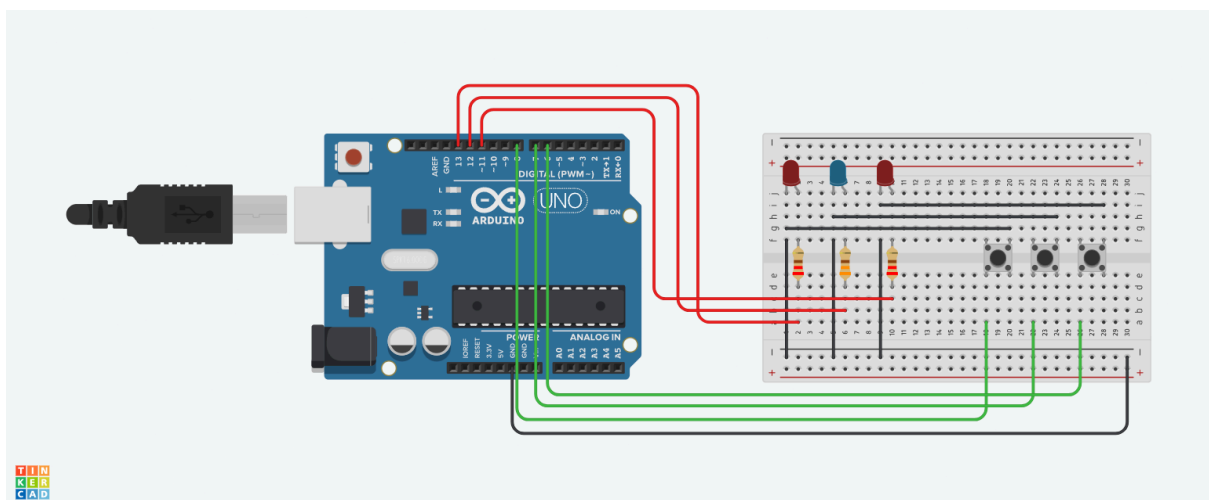
El objetivo de esta sesión fue crear un circuito que contará con tres LEDs y tres pulsadores y, con base a estos componentes, crear un juego en donde uno de los LEDs se prendiera y solo su respectivo pulsador lo pudiera apagar. Una vez el LED apagado, uno de los otros dos se prendería aleatoriamente hasta que su pulsador fuera presionado. Este ciclo se repetiría indefinidamente. En caso de que un pulsador erróneo fuera presionado, el LED activamente prendido seguiría en este estado.

Metodología

Materiales

- Arduino Uno R3 SMD
- Protoboard
- 2 LEDs rojos
- 1 LED azul
- 2 resistores de 220 Ω
- 1 resistor de 330 Ω
- 3 pulsadores
- Cables jumper

Para este circuito se utilizó el Arduino para ejecutar el programa escrito. Por otro lado, el protoboard fue necesario para poder realizar todas las conexiones sin necesidad de soldar ningún componente. Además, para este circuito se utilizaron dos resistencias de 220 Ω y una de 330 Ω (dado que uno de los LEDs era azul); para evitar que los LEDs se quemen. Los tres LEDs y pulsadores se utilizaron para poder cumplir con el objetivo del proyecto que está especificado en la introducción.



Los componentes físicos fueron conectados conforme a la esquemática anterior. Se escogieron seis pines digitales para poder leer y mandar señales a los componentes; específicamente se escogieron los pines 13, 12 y 11 para mandar señales a los LEDs, y para poder leer el estado de los pulsadores se utilizaron los pines 8, 7 y 6. Los pulsadores fueron conectados en modo PULL UP para poder mitigar el uso de resistencias adicionales, puesto que con esta función se activa una resistencia interna del microcontrolador.

Código utilizado

```
1  int leds[] = {13, 12, 11};
2  int botones[] = {8, 7, 6};
3  int ledActual;
4
5  void setup() {
6      for (int i = 0; i < 3; i++) {
7          pinMode(leds[i], OUTPUT);
8      }
9      for (int i = 0; i < 3; i++) {
10         pinMode(botones[i], INPUT_PULLUP);
11     }
12     randomSeed(analogRead(A0));
13     ledActual = random(0, 3);
14     digitalWrite(leds[ledActual], HIGH);
15 }
16
17 void loop() {
18     for (int i = 0; i < 3; i++) {
19         if (digitalRead(botones[i]) == LOW) {
20             if (i == ledActual) {
21                 digitalWrite(leds[ledActual], LOW);
22                 int nuevoLED;
23
24                 do {
25                     nuevoLED = random(0, 3);
26                 } while (nuevoLED == ledActual);
27                 ledActual = nuevoLED;
28                 digitalWrite(leds[ledActual], HIGH);
29                 delay(200);
30             }
31         }
32     }
33 }
```

- Se utilizó inteligencia artificial para auxiliar en el proceso de desarrollo del código.

Explicacion del codigo

```
1  int leds[] = {13, 12, 11};
2  int botones[] = {8, 7, 6};
3  int ledActual;
```

Primeramente, se definen los arreglos y variables que se utilizarán más adelante. Se utilizan arreglos para poder vincular un LED con un pulsador en base a un solo valor, sin importar las diferencias de entradas. De esta forma, si se llama al valor 0 de cada arreglo, se llamará a ambas entradas correctamente (la del LED y la del pulsador). Por otro lado, la variable "ledActual" servirá para trabajar con el LED prendido al momento de lectura.

```
5  void setup() {
6      for (int i = 0; i < 3; i++) {
7          pinMode(leds[i], OUTPUT);
8      }
9      for (int i = 0; i < 3; i++) {
10         pinMode(botones[i], INPUT_PULLUP);
11     }
}
```

Dentro del setup del sketch, se utiliza el operador "void" el cuál indica que las funciones "setup" y "loop" no regresan ningún valor. Posteriormente, se define al arreglo de los leds como valores de salida por medio de "OUTPUT" y el arreglo de los botones como entradas a través de la constante "INPUT_PULLUP".

```
12     randomSeed(analogRead(A0));
13     ledActual = random(0, 3);
14     digitalWrite(leds[ledActual], HIGH);
15 }
```

En esta parte del programa, se genera una semilla aleatoria para su uso posterior en la función "random", la cual se utiliza para que el primer foco que se encienda sea aleatorio. De no ser por la semilla, el primer foco que se prendiera siempre sería el mismo. Una vez el primer foco decidido, este se prende al iniciarse el programa mediante HIGH.

```

17 void loop() {
18     for (int i = 0; i < 3; i++) {
19         if (digitalRead(botones[i]) == LOW) {
20             if (i == ledActual) {
21                 digitalWrite(leds[ledActual], LOW);
22                 int nuevoLED;
23
24                 do {
25                     nuevoLED = random(0, 3);
26                 } while (nuevoLED == ledActual);
27                 ledActual = nuevoLED;
28                 digitalWrite(leds[ledActual], HIGH);
29                 delay(200);
30             }
31         }
32     }
33 }

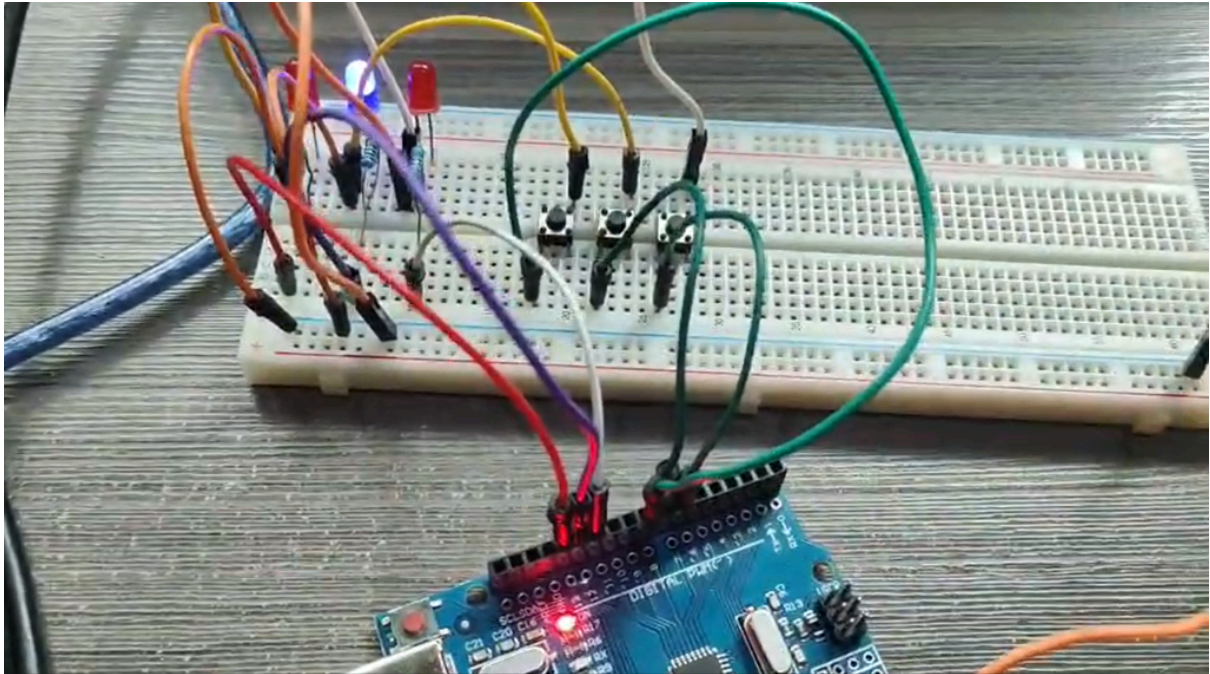
```

Dentro del loop del sketch, el programa revisa una y otra vez el estado de los tres botones para detectar si alguno fue presionado. Cuando se detecta que un botón fue presionado, se compara la posición de presionado con la variable "ledActual", que es el que dice cuál LED se encuentra encendido. Si no se presiona el botón correspondiente al LED que está encendido, las posiciones no coinciden y el programa no hará ningún cambio. Por lo que, el LED actualmente encendido permanece así hasta que se presiona el botón correcto.

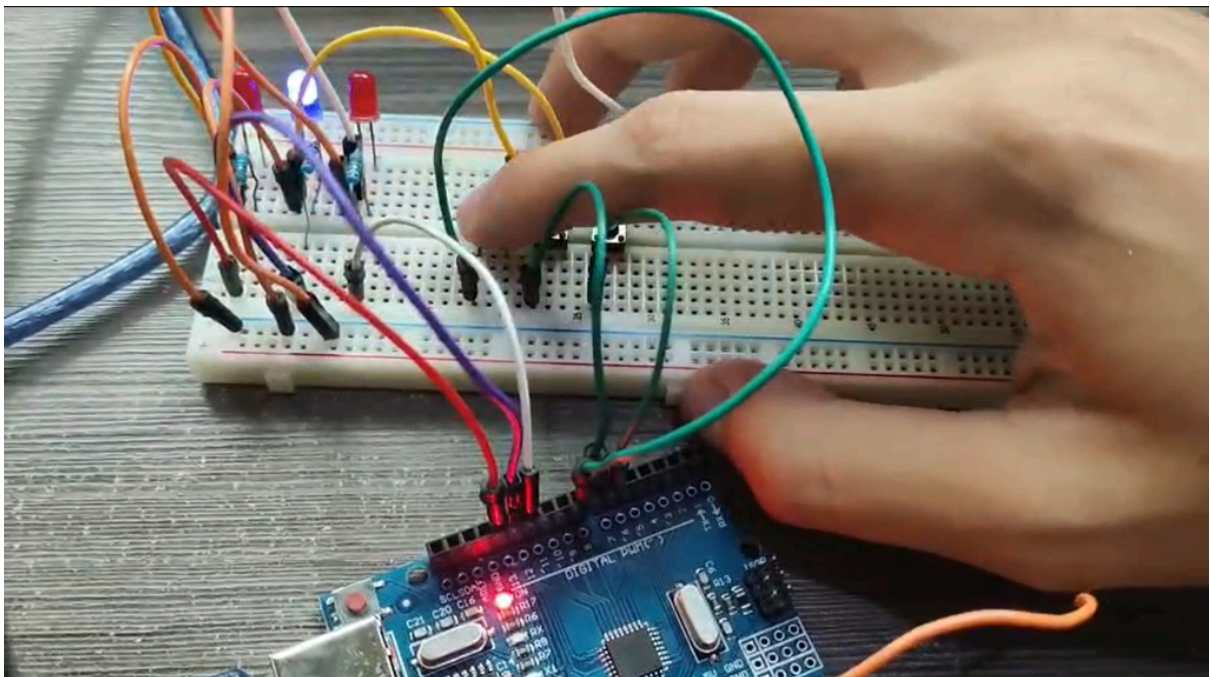
Si coincide, se presionó el botón correcto. En ese caso, el LED se apaga y se genera una nueva variable "nuevoLED". Después de esto, el programa entra en el bucle "do while", el cual genera un valor para "nuevoLED" y checa si es el mismo que el valor pasado de "ledActual". Si el valor es el mismo, se repite el bucle. Si no, se actualiza el valor de "ledActual" y se prende, agregando un pequeño delay. Esto sirve para que el siguiente LED en prenderse nunca sea el mismo al anterior. Después de esto, el programa regresa a checar el estado de los tres botones y esto se repite indefinidamente.

Resultados

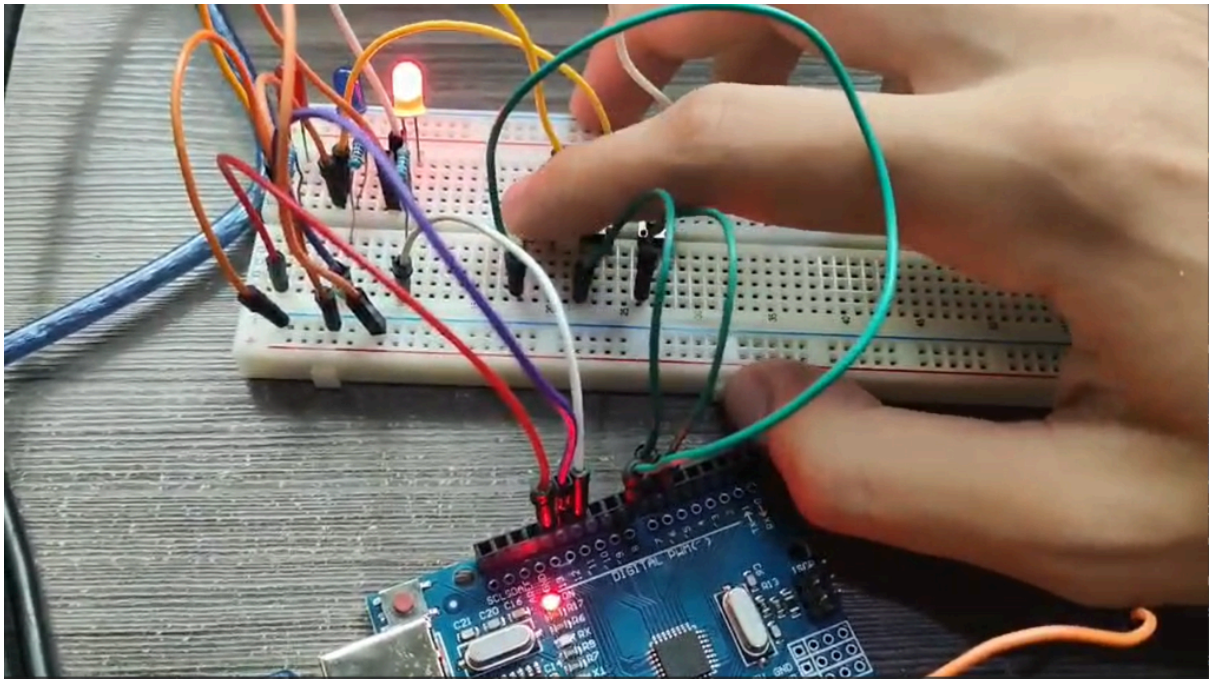
Una vez el sistema conectado y el programa subido al Arduino, se probó su funcionamiento. Los resultados de este dieron lo siguiente:



- Cuando el arduino se encendía, uno de los tres LEDs se enciende aleatoriamente.



- Cuando se presionó un botón que no correspondía al LED encendido, el sistema mantuvo su estado y el LED permanece encendido.



- Cuando se presionó el botón correspondiente al LED encendido, este se apagó. Después, uno de los otros dos LEDs se encendió de manera aleatoria.

Conclusión

En síntesis, el circuito creado dio los resultados esperados. Los tres LEDs se prenden de manera aleatoria una vez que el usuario presiona su pulsador correspondiente; se cumplen totalmente con las especificaciones del proyecto en cuanto a construcción física como en programación.

Sin embargo, cabe mencionar como la construcción del circuito fue considerablemente más complicada que la del anterior avance, puesto que a lo largo del proceso nos encontramos con errores que a simple vista no tienen razón de ocurrir, y la programación demandó de estructuras de control más avanzadas que las vistas previamente.

De cualquier forma, como aprendizaje final de este proyecto obtuvimos un panorama más amplio de las capacidades del arduino y del lenguaje de C++. Además, aprendimos que las esquemáticas no siempre son finales, puesto que en la construcción física las cosas pueden que necesiten ajustes. El desarrollo de máquinas y programas nunca es lineal, ya que a menudo se requieren de múltiples iteraciones para obtener un resultado satisfactorio.

Referencias

OpenAI. (2026, 27 de agosto). *Programar juego de reacción* [Chat de IA generativa].

ChatGPT.

<https://chatgpt.com/share/6a94ecd6-1cac-83e8-bdbd-6b8118fc2b4d>